

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Современные операционные системы и базы данных»
на тему:
**«Диагностика HTTP сервиса средствами операционной
системы»**

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверила
Преподаватель:
доцент, к.н., Галлини Надежда
Игоревна

Оценка: _____
Дата: 17.09.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

1. Цель работы

Цель практической работы — разработать локальный HTTP-сервис, запустить его как отдельный процесс операционной системы и исследовать связь между приложением, процессом, сокетом, TCP-портом и HTTP-ответом. Работа завершается автоматизированной диагностикой нормального режима, тайм-аута, деградации зависимости и недоступного порта.

2. Структура проекта

Исходные файлы размещены в каталоге `app`:

`app/server.py` — учебный HTTP-сервис (`ThreadingHTTPServer`, `127.0.0.1:8080`)

`app/check_service.py` — автоматическая `healthcheck`-проверка (`exit code 0/1`)

`app/tests/test_health.py` — автотесты кодов `200`, `404` и `503`

`app/evidence/` — технические доказательства (скриншоты и дампы ответов)

3. Часть 1. Реализация сервиса

Файл `server.py` использует только стандартную библиотеку Python. `ThreadingHTTPServer` обслуживает запросы в отдельных потоках, поэтому медленный `endpoint /slow` не блокирует остальные запросы.

`GET /` — HTML-страница «Учебный HTTP-сервис работает»

`GET /health` — JSON `status=ok`, `pid`, `port`, `uptime_s` (HTTP 200)

`GET /slow` — задержка 2 с (HTTP 200)

`GET /error` — имитация отказа зависимости (HTTP 503)

прочий путь — JSON `status=not_found` (HTTP 404)

Сервис привязан к `127.0.0.1:8080` (`loopback`). Это ограничивает доступ локальным компьютером. Привязка к `0.0.0.0` потребовала бы отдельного анализа файрвола.

4. Часть 2. Запуск и первичная проверка

Сервис запущен командой `python server.py`. Первичная проверка выполнена через браузер и `curl.exe`.

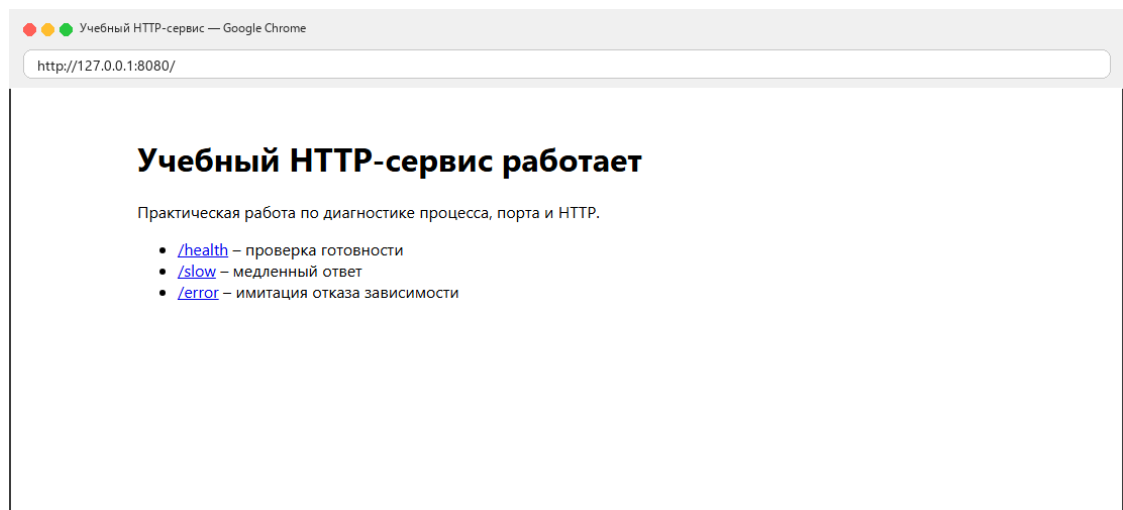


Рисунок 1 – Стартовая страница работающего учебного HTTP-сервиса

```
PowerShell - curl /health

PS C:\Users\mitriy\Desktop\task\app> curl.exe -i http://127.0.0.1:8080/health
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.7
Date: Thu, 17 Sep 2026 12:00:03 GMT
Content-Type: application/json
Content-Length: 61

{"status": "ok", "pid": 7668, "port": 8080, "uptime_s": 10.3}

PS C:\Users\mitriy\Desktop\task\app> python check_service.py
OK: service is healthy
check_exit:0
```

Рисунок 2 – Ответ endpoint /health (200 OK): PID, порт и время работы процесса

Фактический ответ /health: HTTP/1.0 200 OK, Content-Type: application/json, тело {"status": "ok", "pid": 7668, "port": 8080, "uptime_s": ...}. Скрипт check_service.py завершился с кодом 0 и сообщением «OK: service is healthy».

5. Часть 3. Связь процесса и порта

Выполнены команды Get-NetTCPConnection -LocalPort 8080 и Get-Process -Id <OwningProcess>. Сопоставлены три значения: PID в JSON /health = 7668, OwningProcess состояния Listen = 7668, Id процесса python = 7668. Совпадение доказывает связь HTTP-ответа с процессом ОС.

```
PowerShell – порт 8080, PID и режимы отказа

PS C:\...> Get-NetTCPConnection -LocalPort 8080 | Format-Table LocalAddress,LocalPort,State,OwningProcess
LocalAddress LocalPort State OwningProcess
-----
127.0.0.1      8080 TimeWait      0
127.0.0.1      8080 TimeWait      0
127.0.0.1      8080 TimeWait      0
127.0.0.1      8080 Listen        7668

PS C:\...> Get-Process -Id 7668 | Format-List Id,ProcessName,CPU,WorkingSet,StartTime
Id          : 7668
ProcessName : python
CPU         : 0,265625
WorkingSet  : 23470080
StartTime   : 17.09.2026 14:59:53

# Сопоставление: PID из /health = 7668, OwningProcess(Listen) = 7668, Id процесса = 7668

PS C:\...> curl.exe -i http://127.0.0.1:8080/error
HTTP/1.0 503 Service Unavailable
{"status": "degraded", "reason": "dependency unavailable"}

PS C:\...> curl.exe --max-time 1 http://127.0.0.1:8080/slow
curl: (28) Operation timed out after 1009 milliseconds with 0 bytes received
```

Рисунок 3 – Диагностическая сессия: порт 8080 ↔ PID 7668, ответы 503 и timeout

6. Часть 4. Матрица отказов (шесть экспериментов)

Результаты экспериментов зафиксированы командами curl.exe и сохранены в app/evidence/.

Эксперимент	Команда	Результат	Вывод
Нормальная работа	curl.exe -i .../health	200 OK, status=ok, PID=7668	Процесс и endpoint готовы
Неизвестный путь	curl.exe -i .../unknown	404 Not Found	TCP/HTTP работают, ресурса нет
Деградация	curl.exe -i .../error	503 Service Unavailable	Сервис доступен, зависимость нет
Тайм-аут	curl.exe --max-time 1 .../slow	curl error 28	Клиент прекратил ожидание
Неверный порт	curl.exe -i ...:8081/health	connection refused (exit 7)	На порту никто не слушает
Остановленный процесс	stop server.py + /health	connection refused	Недоступен процесс сервера

```
PowerShell – 503 Service Unavailable

PS C:\Users\mitriy\Desktop\task\app> curl.exe -i http://127.0.0.1:8080/error
HTTP/1.0 503 Service Unavailable
Server: BaseHTTP/0.6 Python/3.14.7
Date: Thu, 17 Sep 2026 12:00:04 GMT
Content-Type: application/json
Content-Length: 58

{"status": "degraded", "reason": "dependency unavailable"}
```

Рисунок 4 – Ответ 503 Service Unavailable при деградации зависимости

```
PowerShell - 404 Not Found

PS C:\Users\mitriy\Desktop\task\app> curl.exe -i http://127.0.0.1:8080/unknown
HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.14.7
Date: Thu, 17 Sep 2026 12:00:04 GMT
Content-Type: application/json
Content-Length: 43

{"status": "not_found", "path": "/unknown"}
```

Рисунок 5 – Ответ 404 Not Found для неизвестного пути /unknown

```
PowerShell - timeout /slow

PS C:\Users\mitriy\Desktop\task\app> curl.exe -i --max-time 1 http://127.0.0.1:8080/slow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0      0     0
  0     0    0     0    0     0      0      0      0     0
curl: (28) Operation timed out after 1009 milliseconds with 0 bytes received
EXIT:28
```

Рисунок 6 – Тайм-аут клиента на endpoint /slow (curl error 28)

```
PowerShell - connection refused

PS C:\Users\mitriy\Desktop\task\app> curl.exe -i http://127.0.0.1:8081/health
curl: (7) Failed to connect to 127.0.0.1:8081 after 2031 ms: Could not connect to server
EXIT:7

# Аналогичный результат после остановки server.py на порту 8080
```

Рисунок 7 – Connection refused при обращении к порту 8081

7. Часть 5. HTTP и TCP отвечают на разные вопросы

Ping проверяет достижимость узла средствами ICMP и не проверяет TCP-порт и HTTP-сервис. Успешный ping не доказывает открытость порта 8080. Неуспешный ping не всегда означает недоступность HTTP, так как ICMP может фильтроваться отдельно.

Connection refused означает, что ОС получила попытку соединения, но на целевом порту нет слушающего сокета. Timeout означает отсутствие ответа за заданное время (в эксперименте — 1 с при задержке /slow = 2 с).

Процесс — Get-Process: запущена ли программа и сколько ресурсов использует

TCP — Get-NetTCPConnection: какой PID слушает порт и состояние соединения

HTTP — curl.exe -i: код, заголовки и тело ответа

Прикладной контракт — check_service.py / тесты: поля JSON и exit code

8. Часть 6. Автоматизация healthcheck

Скрипт `check_service.py` обращается к `http://127.0.0.1:8080/health` с тайм-аутом 1 с. Код возврата 0 только при HTTP 200 и `status=ok`. При работающем сервисе получено: «ОК: service is healthy», `exit code` = 0. Ненулевой код используется в CI/оркестраторе для блокировки деплоя или исключения инстанса из балансировки.

9. Часть 7. Дополнительное задание (выводы)

При десяти параллельных запросах к `/slow` `ThreadingHTTPServer` обрабатывает их одновременно: общее время близко к ~ 2 с. Однопоточный `HTTPServer` выполняет запросы последовательно: примерно 10×2 с ≈ 20 с, так как `time.sleep` блокирует единственный поток.

Для доступа с другой машины нужны: привязка не к 127.0.0.1, правила файрвола, аутентификация/TLS. В production-журнал нельзя писать пароли, токены и персональные данные. Автотесты `tests/test_health.py` поднимают сервер на свободном порту и проверяют 200/404/503.

10. Схема прохождения запроса

Клиент `curl` формирует HTTP-запрос $\rightarrow$ стек TCP/IP ОС устанавливает соединение с сокетом 127.0.0.1:8080 $\rightarrow$ процесс Python (PID 7668) принимает соединение в состоянии Listen $\rightarrow$ HTTP-сервис выполняет endpoint $\rightarrow$ клиент получает код, заголовки и тело ответа.

Клиент (curl) $\rightarrow$ TCP/IP стек ОС $\rightarrow$ Сокет :8080 $\rightarrow$ Процесс Python (PID 7668) $\rightarrow$ Endpoint

11. Выводы

1. Программа, процесс, поток, сокет и порт — разные уровни абстракции ОС и сети.

2. Связь порта 8080 с процессом доказана: PID /health = `OwningProcess` = `Id` = 7668.

3. Воспроизведены режимы 200, 404, 503, `timeout` и `connection refused`; различия объяснены.

4. Healthcheck возвращает корректный `exit code` для автоматизации.

5. Многопоточный сервер не блокирует параллельные /slow; однопоточный — блокирует.

12. Ответы на контрольные вопросы

1. PID назначается ОС при создании процесса; после перезапуска создаётся новый процесс с другим PID.

2. Порт — число в транспортном заголовке; сокет — объект ОС/приложения, связанный с адресом и портом.

3. Listen доказывает, что процесс ожидает входящие соединения на данном порту.

4. Ping проверяет ICMP-достижимость узла, а не TCP-порт и не HTTP.

5. Connection refused — соединение отклонено (нет слушателя); timeout — ответа не было вовремя.

6. 503 приходит после успешного TCP и разбора HTTP: транспорт работает, прикладной статус — ошибка зависимости.

7. Ненулевой код healthcheck в CI/оркестраторе останавливает деплой или убирает инстанс из балансировки.

8. Однопоточный сервер во время /slow не обслуживает другие запросы до конца sleep.

9. 0.0.0.0 открывает сервис на всех интерфейсах — риск внешнего доступа без защиты.

10. В журнал нельзя писать пароли, ключи API, токены, персональные данные и иные секреты.

13. Технические доказательства

Все скриншоты и текстовые дампы сохранены в каталоге app/evidence/. Сервис запускался на 127.0.0.1:8080; PID процесса python — 7668. Дата эксперимента: 17.09.2026.